

Go Toolbox

Table of Contents

Packages	1
Wrapper	1
Result	2
Tuple	2
Optional	2
Must	2
Data Structures	2
Set	2
Ordered	3
Iterator	3
Object	3
Concurrency	3
Pool	3

A collection of packages for Go.

The toolbox is a place where I collect stuff I either used and implemented multiple times already or I want to try out and don't have another place for it yet.

I try to keep everything tested and documented, but I don't guarantee to keep packages in here maintained.

If you find something of it useful, I recommend having a look on the implementation and do something similar on your own. According to the go proverb:

A little copying is better than a little dependency.

— Rob Pike in his 'Go Proverbs' talk, <https://go-proverbs.github.io>

Packages

Currently, the tool box contains the following packages, more may follow.

If some of them turns out specifically useful, they may be moved to their own modules.

Wrapper

Types for wrapping values together for specific use cases.

Result

Package result provides a type wrapping either a value or an error. It is inspired by rusts result type, but tries to remain as idiomatic to go as possible

Main use case is to send it over channels, when possible errors should be propagated to the receiver.

Apart from that, after calling `Get()`, it can be used with go's idiomatic error handling, like:

```
value, err := result.Get()
if err != nil {
    //handle error
}

// Do something with value
```

Tuple

Package tuple provides simple wrappers for specific use cases.

Optional

Optional provides a generic Optional type for Go.

For a lot of use cases, I don't like mixing the concepts of referencing and optionality, as it happens with pointers, so this provides an alternative.

The Optionals implemented in this package handle marshalling and unmarshalling to/from json and xml and are handling null-values, not existing values and omitzero/omitempty.

Must

Package must provides functions which panics on error and returns the value(s) passed to them.

This is usually not recommended, but can be helpful in tests or initialization of an application.

It can be used like `regex.MustCompile` or `template.MustParse` but for arbitrary functions (With two or three return parameters).

Data Structures

Packages implementing or working with more complex data structures.

Set

Package set provides an implementation of a set, meaning a collection of unique, unordered values.

It implements basic collection options as add, remove, contains, len and clear as well as common

set operations like union, intersection and difference.

Ordered

Package ordered provides ordered map and set implementations that maintain insertion order.

They provide $O(1)$ time complexity operations while allowing iterating over the elements in the order they were added.

Iterator

Package iterator implements composable iterator functions.

Currently ,this is mainly for playing around and getting to know the characteristics of the go iterators, which are part of the language since go 1.23 (when writing this, go 1.24 is the recent version, so they are still quite new).

Object

Package object provides utility methods to work with values where the underlying type is 'map[string]any'.

This can be handy when working with unknown or dynamic json/yaml/toml/... data.

Concurrency

Packages implementing helpers for concurrent programming.

Pool

Package pool provides a convenient way to create a worker pools